

程序设计 II 第 14 周 题目汇总（完整版）

来源：Matrix · 课程「程序设计 II（25 级 曾坤 计算机学院）」含：完整题面、提交文件、平台提供的初始文件（.hpp/main.cpp 等）。选择题含标准答案。

共 7 项

25 程设 II-周 14-课后 1

类型：编程题

题面

仓库货物管理系统

管理三种货物，类层次：Cargo（抽象基类）→ Electronics(final)、Furniture、Clothing。

```
class Cargo {
public:
    virtual string name() = 0;
    virtual int weight() = 0;
    virtual int value() = 0;
    virtual ~Cargo() {}
};
```

各货物信息：Electronics(5,500)、Furniture(20,150)、Clothing(2,100)。

操作：ADD type、WEIGHT（总重量）、TOTAL（总价值）、COUNT type（某类型数量）、IDENTIFY index（第 index 件 1-based 的 typeid(*p).name()）、EXIT。

输入样例

```
11
ADD Electronics
ADD Furniture
ADD Clothing
ADD Electronics
WEIGHT
TOTAL
COUNT Electronics
COUNT Furniture
COUNT Clothing
IDENTIFY 1
IDENTIFY 3
EXIT
```

输出样例

```
32
1250
2
1
1
11Electronics
8Clothing
```

提交文件：Cargos.h、Cargos.cpp

平台提供的初始文件

文件 Cargo.h：

```
#ifndef CARGO_H
#define CARGO_H

#include <string>
using namespace std;

class Cargo {
public:
    virtual string name() = 0;
    virtual int weight() = 0;
    virtual int value() = 0;
    virtual ~Cargo() {}
};

#endif
```

文件 main.cpp：

```
#include <iostream>
#include <vector>
#include <unordered_map>
#include <typeinfo>
#include "Cargos.h"
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    vector<Cargo*> warehouse;
    unordered_map<string, int> cnt;
    int totalWeight = 0;
    int totalValue = 0;
```

```

for (int i = 0; i < n; i++) {
    string op;
    cin >> op;

    if (op == "ADD") {
        string t;
        cin >> t;

        Cargo* p = nullptr;
        if (t == "Electronics") p = new Electronics();
        else if (t == "Furniture") p = new Furniture();
        else if (t == "Clothing") p = new Clothing();

        warehouse.push_back(p);
        cnt[t]++;
        totalWeight += p->weight();
        totalValue += p->value();
    }
    else if (op == "WEIGHT") {
        cout << totalWeight << '\n';
    }
    else if (op == "TOTAL") {
        cout << totalValue << '\n';
    }
    else if (op == "COUNT") {
        string t;
        cin >> t;
        cout << cnt[t] << '\n';
    }
    else if (op == "IDENTIFY") {
        int idx;
        cin >> idx;
        Cargo* p = warehouse[idx - 1];
        cout << typeid(*p).name() << '\n';
    }
    else if (op == "EXIT") {
        break;
    }
}

for (auto p : warehouse)
    delete p;

return 0;
}

```

25 程设 II-周 14-课后 2

类型：编程题

题面

餐饮管理系统

管理三种餐品，类层次：Food（抽象基类）→ MainCourse、Dessert、Drink。

```
class Food {  
public:  
    virtual string name() = 0;  
    virtual int price() = 0;  
    virtual void serve() = 0;  
    virtual ~Food() {}  
};
```

各餐品信息：MainCourse(30,“Serving main course!”)、Dessert(15,“Serving dessert!”)、Drink(10,“Serving drink!”)。

操作：ADD type、SERVE（按 ADD 顺序调用所有 serve()）、TOTAL（总价格）、COUNT type、EXIT。

输入样例

```
10  
ADD MainCourse  
ADD Dessert  
ADD Drink  
ADD MainCourse  
SERVE  
TOTAL  
COUNT MainCourse  
COUNT Dessert  
COUNT Drink  
EXIT
```

输出样例

```
Serving main course!  
Serving dessert!  
Serving drink!  
Serving main course!  
85  
2  
1  
1
```

提交文件：Foods.h、Foods.cpp

平台提供的初始文件

文件 Food.h :

```
#ifndef FOOD_H
#define FOOD_H

#include <string>
using namespace std;

class Food {
public:
    virtual string name() = 0;
    virtual int price() = 0;
    virtual void serve() = 0;
    virtual ~Food() {}
};

#endif
```

文件 main.cpp :

```
#include <iostream>
#include <vector>
#include <unordered_map>
#include "Foods.h"
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    vector<Food*> menu;
    unordered_map<string, int> cnt;
    int total = 0;

    for (int i = 0; i < n; i++) {
        string op;
        cin >> op;

        if (op == "ADD") {
            string t;
            cin >> t;

            Food* p = nullptr;
            if (t == "MainCourse") p = new MainCourse();
            else if (t == "Dessert") p = new Dessert();
```

```

        else if (t == "Drink") p = new Drink();

        menu.push_back(p);
        cnt[t]++;
        total += p->price();
    }
    else if (op == "SERVE") {
        for (auto p : menu)
            p->serve();
    }
    else if (op == "TOTAL") {
        cout << total << '\n';
    }
    else if (op == "COUNT") {
        string t;
        cin >> t;
        cout << cnt[t] << '\n';
    }
    else if (op == "EXIT") {
        break;
    }
}

for (auto p : menu)
    delete p;

return 0;
}

```

25 程设 II-周 14-课后 3

类型：编程题

题面

图形接口系统

抽象接口 Shape（纯虚 area/name/scale + 虚析构）+ 多继承接口 Printable（纯虚 print）。派生 Circle/Rectangle/Square（均 final，多继承 Shape 和 Printable）。

Circle 面积 πr^2 ；Rectangle $w \times h$ ；Square a^2 。

操作：ADD Circle r/ADD Rectangle w h/ADD Square a、SUM（面积总和）、COUNT type（运行时精确类型匹配）、MAX/MIN（基于 area，空容器输出 0.00）、SCALE k（尺寸×k）、REMOVE type（精确类型删除并 delete）、PRINT（按序输出，空容器输出 NO）、over 结束。

注意：const double PI = 3.1415926; ; COUNT/REMOVE 用 typeid(*obj)==typeid(T) 或 dynamic_cast，禁止 typeid(x).name() 字符串比较；name() 严格大小写匹配。

输入样例

```
ADD Circle 1
ADD Rectangle 2 3
ADD Square 2
SUM
COUNT Circle
MAX
REMOVE Rectangle
SUM
over
```

输出样例

```
13.14
1
6.00
7.14
```

提交文件：shapes.cpp

平台提供的初始文件

文件 shape.h：

```
#ifndef SHAPE_H
#define SHAPE_H

#include <string>

class Shape {
public:
    virtual double area() const = 0;
    virtual std::string name() const = 0;
    virtual void scale(double k) = 0;
    virtual ~Shape() {}
};

#endif
```

文件 main.cpp：

```
#include <iostream>
#include <iomanip>
#include "shapes.h"

using namespace std;

int main() {
```

```

ios::sync_with_stdio(false);
cin.tie(nullptr);

string op;

while (cin >> op && op != "over") {

    if (op == "ADD") {
        string type;
        cin >> type;

        if (type == "Circle") {
            double r;
            cin >> r;
            shapes.push_back(new Circle(r));
        }
        else if (type == "Rectangle") {
            double w, h;
            cin >> w >> h;
            shapes.push_back(new Rectangle(w, h));
        }
        else if (type == "Square") {
            double a;
            cin >> a;
            shapes.push_back(new Square(a));
        }
    }

    else if (op == "SUM") {
        cout << fixed << setprecision(2) << sumArea() << "\n";
    }

    else if (op == "COUNT") {
        string t;
        cin >> t;
        cout << countType(t) << "\n";
    }

    else if (op == "MAX") {
        cout << fixed << setprecision(2) << maxArea() << "\n";
    }

    else if (op == "MIN") {
        cout << fixed << setprecision(2) << minArea() << "\n";
    }

    else if (op == "SCALE") {
        double k;
        cin >> k;
    }
}

```



```

    for (auto s : shapes) s->scale(k);
}

else if (op == "REMOVE") {
    string t;
    cin >> t;
    removeType(t);
}

else if (op == "PRINT") {
    if (shapes.empty()) {
        cout << "NO\n";
        continue;
    }

    for (auto s : shapes) {
        if (auto p = dynamic_cast<Printable*>(s)) {
            p->print();
        }
    }
}

for (auto s : shapes) delete s;
return 0;
}

```

文件 shapes.h :

```

#ifndef SHAPES_H
#define SHAPES_H

#include <vector>
#include <string>
#include "shape.h"

class Printable {
public:
    virtual void print() const = 0;
    virtual ~Printable() {}
};

class Circle final : public Shape, public Printable {
private:
    double r;

public:
    explicit Circle(double r);

    double area() const override;

```

```

    std::string name() const override;
    void scale(double k) override;
    void print() const override;
};

class Rectangle final : public Shape, public Printable {
private:
    double w, h;

public:
    Rectangle(double w, double h);

    double area() const override;
    std::string name() const override;
    void scale(double k) override;
    void print() const override;
};

class Square final : public Shape, public Printable {
private:
    double a;

public:
    explicit Square(double a);

    double area() const override;
    std::string name() const override;
    void scale(double k) override;
    void print() const override;
};

extern std::vector<Shape*> shapes;

void removeType(const std::string& t);
int countType(const std::string& t);
double sumArea();
double maxArea();
double minArea();

#endif

```

25 程设 II-周 14-课堂 1

类型：编程题

题面

交通工具管理系统

类层次：Vehicle（抽象基类）－ Car(final)、Bicycle（move() 为 final）、Boat。

```
class Vehicle{
public:
    virtual string type() = 0;
    virtual int speed() = 0;
    virtual void move() = 0;
    virtual ~Vehicle() {}
};
```

信息：Car(120,“Car is driving!”)、Bicycle(20,“Bicycle is cycling!”)、Boat(60,“Boat is sailing!”)。

操作：ADD type、MOVE、SPEED（最高速度）、IDENTIFY index
（typeid(*p).name()）、UPGRADE index（dynamic_cast 转 Car，成功 “Upgraded: Car”
失败 “Not a Car”）、EXIT。

输入样例

```
10
ADD Car
ADD Bicycle
ADD Boat
MOVE
SPEED
IDENTIFY 1
IDENTIFY 2
UPGRADE 1
UPGRADE 3
EXIT
```

输出样例

```
Car is driving!
Bicycle is cycling!
Boat is sailing!
120
3Car
7Bicycle
Upgraded: Car
Not a Car
```

提交文件：Vehicles.h、Vehicles.cpp

平台提供的初始文件

文件 Vehicle.h：

```

#ifndef VEHICLE_H
#define VEHICLE_H

#include <string>
using namespace std;

class Vehicle {
public:
    virtual string type() = 0;
    virtual int speed() = 0;
    virtual void move() = 0;
    virtual ~Vehicle() {}
};

#endif

```

文件 main.cpp :

```

#include <iostream>
#include <vector>
#include <typeinfo>
#include <algorithm>
#include "Vehicles.h"
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    vector<Vehicle*> garage;

    for (int i = 0; i < n; i++) {
        string op;
        cin >> op;

        if (op == "ADD") {
            string t;
            cin >> t;

            Vehicle* p = nullptr;
            if (t == "Car") p = new Car();
            else if (t == "Bicycle") p = new Bicycle();
            else if (t == "Boat") p = new Boat();

            garage.push_back(p);
        }
    }
}

```

```

else if (op == "MOVE") {
    for (auto p : garage)
        p->move();
}
else if (op == "SPEED") {
    int maxSpeed = 0;
    for (auto p : garage)
        maxSpeed = max(maxSpeed, p->speed());
    cout << maxSpeed << endl;
}
else if (op == "IDENTIFY") {
    int idx;
    cin >> idx;
    Vehicle* p = garage[idx - 1];
    cout << typeid(*p).name() << endl;
}
else if (op == "UPGRADE") {
    int idx;
    cin >> idx;
    Vehicle* p = garage[idx - 1];
    Car* c = dynamic_cast<Car*>(p);
    if (c != nullptr)
        cout << "Upgraded: Car" << endl;
    else
        cout << "Not a Car" << endl;
}
else if (op == "EXIT") {
    break;
}
}

for (auto p : garage)
    delete p;

return 0;
}

```

25 程设 II-周 14-课堂 2

类型：编程题

题面

动物园管理系统

抽象类 Animal（纯虚 speak/getFood/getType + 虚析构）
 ◦ Dog(Woof,5)/Cat(Meow,3)/Cow(Moo,10)/Sheep(Baa,7) ◦

操作：ADD type、SPEAK（按加入顺序发声）、FEED（每日总食物消耗）、COUNT type、TYPECHECK（用 RTTI 统计 Dog:x Cat:x Cow:x Sheep:x）、EXIT。

输入样例

```
8
ADD Dog
ADD Cat
ADD Cow
COUNT Dog
FEED
TYPECHECK
SPEAK
EXIT
```

输出样例

```
1
18
Dog:1
Cat:1
Cow:1
Sheep:0
Woof
Meow
Moo
```

提交文件：main.cpp

（本题无平台初始文件）

25 程设 II-周 14-课堂 3

类型：编程题

题面

电子设备资产管理系统

抽象基类 Device（纯虚 getPower/getLife + 虚析构）+ 接口 Usable（纯虚 use）。派生 Laptop(60,180)/Desktop(120,360)/Printer(80,240)（均 final，多继承 Device+Usable，用 override），统一存 vector<Device*>。

操作：PURCHASE type、TOTAL（总功耗，空 0）、COUNT_N type（数量，必须 dynamic_cast）、COUNT_L type（剩余寿命总和，输出 type value）、MAX_N/MIN_N/MAX_L/MIN_L（空输出 NO，平局按 Laptop>Desktop>Printer）、USE type（对该类型 life 最大的设备执行 life-=power，life≤0 删除释放；空或无该类型输出 NO）、TYPECHECK（typeid 统计，空输出 NO）、END。

输入样例

```
PURCHASE Laptop
PURCHASE Laptop
PURCHASE Desktop
PURCHASE Printer
TOTAL
COUNT_N Laptop
COUNT_L Laptop
USE Laptop
COUNT_L Laptop
TYPECHECK
MAX_N
MIN_L
END
```

输出样例

```
320
2
360
300
Laptop:2
Desktop:1
Printer:1
Laptop 2
Printer 240
```

提交文件：devices.cpp

平台提供的初始文件

文件 main.cpp：

```
#include <iostream>
#include "devices.h"
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string op;
    while (cin >> op && op != "END") {
        if (op == "PURCHASE") {
            string t; cin >> t;
            if (t == "Laptop") devices.push_back(new Laptop());
            else if (t == "Desktop") devices.push_back(new Desktop());
            else devices.push_back(new Printer());
        }
        else if (op == "TOTAL") cout << totalPower() << "\n";
        else if (op == "COUNT_N") { string t; cin >> t; cout << countN(t) << "\n"; }
```

```

        else if (op == "COUNT_L") { string t; cin >> t; cout << countL(t) << "\n"; }
        else if (op == "MAX_N") cout << maxN() << "\n";
        else if (op == "MIN_N") cout << minN() << "\n";
        else if (op == "MAX_L") cout << maxL() << "\n";
        else if (op == "MIN_L") cout << minL() << "\n";
        else if (op == "USE") { string t; cin >> t; use(t); }
        else if (op == "TYPECHECK") typeCheck();
    }

    for (auto d : devices) delete d;
    return 0;
}

```

文件 device.h :

```

#ifndef DEVICE_H
#define DEVICE_H

#include <string>

class Device {
public:
    virtual int getPower() const = 0;
    virtual int getLife() const = 0;
    virtual std::string name() const = 0;
    virtual ~Device() {}
};

class Usable {
public:
    virtual void use() = 0;
    virtual ~Usable() {}
};

#endif

```

文件 devices.h :

```

#ifndef DEVICES_H
#define DEVICES_H

#include <vector>
#include "device.h"

class Laptop final : public Device, public Usable {
    int power = 60, life = 180;
public:
    int getPower() const override { return power; }
    int getLife() const override { return life; }
    std::string name() const override { return "Laptop"; }
}

```



```

    void use() override { life -= power; }
};

class Desktop final : public Device, public Usable {
    int power = 120, life = 360;
public:
    int getPower() const override { return power; }
    int getLife() const override { return life; }
    std::string name() const override { return "Desktop"; }
    void use() override { life -= power; }
};

class Printer final : public Device, public Usable {
    int power = 80, life = 240;
public:
    int getPower() const override { return power; }
    int getLife() const override { return life; }
    std::string name() const override { return "Printer"; }
    void use() override { life -= power; }
};

extern std::vector<Device*> devices;

int totalPower();
int countN(const std::string& t);
int countL(const std::string& t);
std::string maxN();
std::string minN();
std::string maxL();
std::string minL();
void use(const std::string& t);
void typeCheck();

#endif

```

25 程设II-周 14-理论题

类型：选择题（20 题）

第 1 题

关于多态，下列说法正确的是

- A. 一个对象同时占用多个内存空间
- B. 同一个接口表现出不同的行为
- C. 一个类只能有一个父类
- D. 一个函数只能被调用一次

答案：B

第 2 题

下列关于纯虚函数的声明正确的是

- A. virtual void f();
- B. void f() = 0;
- C. virtual void f() = 0;
- D. void virtual f();

答案：C

第 3 题

关于抽象类，错误的是（ ）

- A. 可以声明指针
- B. 可以声明引用
- C. 可以实例化对象
- D. 可以作为基类

答案：C

第 4 题

当 Animal 为抽象类时，下列代码中合法的是

- A. Animal a;
- B. Animal* p;
- C. Animal x = Dog();
- D. Animal* fun();

答案：B,D

第 5 题

关于虚析构函数的说法正确的是

- A. 析构函数不能是虚函数
- B. 只有派生类需要虚析构
- C. 多态基类应该具有虚析构函数
- D. 虚析构会阻止对象析构

答案：C

第 6 题

dynamic_cast 向下转换失败时，若目标类型是指针，则返回

- A. 0
- B. nullptr
- C. false
- D. 异常

答案：B

第 7 题

关于纯虚函数，下列说法错误的是

- A. 使用 =0 声明
- B. 所在类一定是抽象类
- C. 纯虚函数必须实现函数体
- D. 子类可将其实现为普通函数

答案：C

第 8 题

关于接口风格抽象类，下列说法错误的是

- A. 通常由纯虚函数组成
- B. 可以包含静态成员
- C. 可以作为多重继承的公共接口
- D. 可以包含普通数据成员

答案：D

第 9 题

RTTI 机制主要解决的问题是

- A. 编译时类型检查
- B. 运行时识别对象真实类型
- C. 动态分配内存
- D. 函数重载解析

答案：B

第 10 题

关于抽象类的描述，正确的是

- A. 抽象类可以直接创建对象
- B. 抽象类不能作为基类
- C. 抽象类定义或继承了至少一个纯虚函数
- D. 抽象类不能声明指针

答案：C

第 11 题

关于 final 说明符，下列说法正确的是

- A. final 可以修饰任意成员函数，包括非虚函数
- B. struct B final : A {} 表示 B 不能被任何类继承
- C. final 修饰的函数在子类中仍然可以被覆写
- D. final 是 C++98 引入的关键字

答案：B

第 12 题

以下代码中，哪一行会导致编译错误？

```
struct Base { virtual void foo(); }; // ①
struct A : Base {
    void foo() final; // ②
    void bar() final; // ③
};
struct B final : A {
    void foo() override; // ④
};
struct C : B {}; // ⑤
```

- A. ①
- B. 只有③
- C. ③ 和 ④ 和 ⑤
- D. ②

答案：C

第 13 题

关于 typeid 运算符，下列说法正确的是

- A. 使用 typeid 无需包含任何头文件
- B. 对多态类型对象使用 typeid 时，返回的是对象的静态类型信息
- C. typeid 返回 std::type_info 对象，可通过 name() 获取类型名
- D. typeid 只能用于表达式，不能用于类型名

答案：C

第 14 题

关于 dynamic_cast，下列说法错误的是

- A. dynamic_cast 可以沿继承层级向上、向下及侧向转换

- B. 转型目标为指针类型且转型失败时，返回空指针
- C. 转型目标为引用类型且转型失败时，返回空引用
- D. `dynamic_cast` 是运行时类型识别（RTTI）机制的一部分

答案：C

第 15 题

关于 RTTI 机制，下列说法正确的是

- A. RTTI 是 C 语言的特性，C++ 继承了该机制
- B. RTTI 允许程序在运行时通过基类指针检查所指对象的实际派生类型
- C. RTTI 只提供 `typeid` 一种操作符
- D. RTTI 在编译期就能确定对象的动态类型

答案：B

第 16 题

关于抽象类与接口，下列说法正确的是

- A. C++ 有专门的 `interface` 关键字来定义接口
- B. C++ 可以用所有成员函数均为纯虚函数的类来模拟接口
- C. 接口风格的抽象类不能包含任何静态成员
- D. 接口风格的抽象类必须有构造函数

答案：B

第 17 题

以下关于接口风格抽象类被多重继承的说法，正确的是

- A. 接口风格抽象类多重继承时必然产生命名冲突
- B. 接口风格抽象类因为没有数据成员，可以无歧义地被多重继承
- C. C++ 中多重继承接口风格抽象类需要使用 `implements` 关键字
- D. 接口风格抽象类不能被虚继承

答案：B

第 18 题

下列哪种情况下，抽象类的子类仍然是抽象类？

- A. 子类只覆写了部分纯虚函数，还有纯虚函数未被覆写
- B. 子类覆写了基类所有纯虚函数
- C. 子类新增了自己的普通成员函数
- D. 子类新增了自己的数据成员

答案：A

第 19 题

下列客户端代码中，合法的是（Shape 为抽象类）

```
Shape fun1(int);    // ①  
void fun2(Shape a); // ②  
Shape& fun3(Shape& a); // ③  
Shape x;            // ④  
Shape* p;           // ⑤
```

- A. ③ 和 ⑤
- B. ④ 和 ⑤
- C. ① 和 ②
- D. 只有 ①

答案：A

第 20 题

关于虚析构函数，下列说法正确的是

- A. 只要基类有虚函数，不需要虚析构函数也能正确释放派生类对象
- B. 派生类析构函数必须显式声明 virtual 才能被正确调用
- C. 虚析构函数不能有函数体，必须写成纯虚函数
- D. 任何包含虚函数的基类，其析构函数必须为公开且虚，或受保护且非虚

答案：D
